Non-Convex Polygon Packing via Simulated Annealing and Bracketing-Bisection Search

Joel Solis*

April 2026

Abstract. Take a fixed non-convex polygon. The problem, featured in a 2024 Kaggle competition, is to find the minimum side length L^* of a square container that can hold N congruent copies without overlap, each free to translate and rotate. Answering this exactly is NP-hard. We describe a practical solver combining time-budgeted Simulated Annealing with an outer bisection loop that narrows the bracket containing L^* to a relative precision of 10^{-3}, followed by a feasibility-polish stage. Collision detection uses the Separating Axis Theorem (SAT), chosen because it returns penetration depth, converting a combinatorial feasibility question into a smooth, navigable energy function. We deployed embarrassingly parallel jobs on a five-node HPC cluster for $N \in \{5, 10, 25, 50, 100, 200\}$, with wall times from 3 minutes to 8 hours. Packing densities range from $\eta \approx 0.58$ at small N to $\eta \approx 0.48$ at $N = 200$, with a $1/\sqrt{N}$ boundary-effect law supported by a geometric argument.

Keywords. polygon packing; simulated annealing; separating axis theorem; bisection search; embarrassingly parallel HPC; packing density scaling.

1. INTRODUCTION

Take a fixed non-convex polygon, a 15-vertex shape that appeared in a 2024 Kaggle competition [**?**]. Make N identical copies. The problem is to find the smallest square that holds all of them without overlap. As N grows, the configuration space expands exponentially and exact enumeration becomes hopeless. Even for the simpler case of unit disks the placement decision problem is NP-complete [**?**]: if the decision version of a problem is NP-complete then its optimization version is NP-hard, and ours is at least as hard. Non-convexity adds a further layer of difficulty. It makes collision detection more expensive, the energy landscape more rugged, and the Minkowski sum characterizing pairwise overlap itself non-convex and potentially disconnected.

Exact characterizations via the No-Fit Polygon (NFP), surveyed by Bennell and Oliveira [**?**], require convex decomposition for non-convex shapes and become impractical at the scales we study. Penalty-based SA consistently outperforms approaches restricted to strictly feasible states: as Leung et al. [**?**] showed on orthogonal packing, hard feasibility constraints fragment the search space into disconnected islands, while continuous penalties create traversable bridges between them. Genetic algorithms can explore structurally diverse configurations simultaneously [**?**], and Parallel Tempering [**?**] mixes exponentially faster than single-chain SA on multimodal landscapes, but both require substantially longer budgets than we had available. As Sokal put it: "Monte Carlo is an extremely bad method; it should be used only when all alternative methods are worse." That is the situation here.

*Department of Mathematics, University of Arizona. jsolis@math.arizona.edu. Supported by allocation ece569 on the University of Arizona HPC cluster.

2. PROBLEM FORMULATION

Let be a fixed non-convex polygon with 15 vertices and area $A_p = 0.245625$, computed by the shoelace formula from the reference vertex array. Each copy i is placed by specifying its centre (x_i, y_i) and rotation angle $\theta_i \in [0, 2\pi)$; world vertices follow as $w_k^{(i)} = R(\theta_i)\, v_k + c_i$. A configuration is *feasible* if no two copies overlap and every copy lies within $\Omega_L = [-L/2, L/2]^2$. The goal is to find, for each N, the smallest L admitting a feasible configuration.

Since all N copies must fit inside Ω_L without overlap, the square area satisfies $L^2 \geq NA_p$, giving the hard lower bound $L_{\text{area}} = \sqrt{NA_p}$; the bisection loop never probes below this. Restricting the search to strictly non-overlapping configurations at all times confines it to disconnected feasible islands in $\mathbb{R}^{2N} \times [0, 2\pi)^N$. Allowing temporary, penalized violations creates navigable paths between those islands, which is why the choice of a continuous collision metric rather than a binary one matters so much.

3. ALGORITHM

3.1. Collision Detection Pipeline

Collision detection is invoked after every proposed move to evaluate the energy change, making it the dominant runtime cost. Naive pairwise checking costs $O(N^2)$ per iteration. A uniform spatial hash grid with cell size $h = 2r_{\text{br}} = 1.6$ (twice the bounding radius $r_{\text{br}} = 0.8$) restricts each query to a 5×5 neighbourhood of $\approx 261\eta$ candidates, where $\eta = NA_p/L^2$ is the packing density. The energy change is then computed incrementally, touching only pairs involving the moved polygon, reducing the effective per-move cost to $O(1)$. Of those candidates, roughly 96% are eliminated by polygon-level axis-aligned bounding box (AABB) checks; group-level and per-triangle AABB checks prune the remainder before the Separating Axis Theorem (SAT) is invoked.

Table 1. SA schedule parameters (per trial).

Parameter	Value
Proposals per trial K	100,000
T_{start}	1.0
T_{end}	10^{-5}
Temperature schedule	geometric decay
Step adapt window	2,000 iterations
Acceptance target band	[0.40, 0.60]
Step shrink / grow factor	0.95 / 1.05
Overlap penalty power p	2 (quadratic, fixed)
Feasibility threshold	$< 10^{-6}$

The polygon is triangulated offline into $T = 13$ triangles, a fan from vertex 0, grouped into three clusters of sizes $(5,5,3)$. SAT projects a pair of triangles onto six edge-normal axes and computes the minimum overlap width $d \geq 0$. If any axis shows a gap the triangles are separated; when $d > 0$ it is the *penetration depth*, a continuous measure of overlap severity. World-space vertex coordinates are projected directly onto each axis, avoiding per-call recomputation of rotation matrices. A binary test would assign equal cost to a barely touching pair and a deeply embedded one, yielding a flat energy surface with no gradient to follow. The depth d provides a directional signal, and feeding d^2 into the energy function creates the smooth guidance that steers SA toward feasibility from arbitrary starting configurations.

3.2. Energy Function and SA Schedule

The total energy of a configuration at container side L is

$$E = \lambda \sum_{i<j} \sum_{(a,b)\in\ _{ij}} d(a,b)^2 + \mu \sum_i \phi_i(L), \tag{1}$$

where $d(a,b)$ is the SAT penetration depth between triangle pair (a,b), $\phi_i(L)$ is the squared AABB boundary violation of copy i, and (λ,μ) are penalty weights. Both terms are quadratic in the geometric violation. A configuration is declared feasible when the unweighted residual $= \sum_{i<j} \text{overlap}_{ij} + \sum_i \phi_i < 10^{-6}$.

Each SA trial is time-budgeted rather than iteration-counted. Temperature decreases geometrically, $T_k = T_0 \beta^k$ with $\beta = (T_{\text{end}}/T_0)^{1/K}$, from $T_0 = 1.0$ to $T_{\text{end}} = 10^{-5}$ over $K = 100{,}000$ proposals. Step sizes (s_{xy}, s_θ) adapt every 2,000 iterations: if acceptance falls below 40% they shrink by factor 0.95; above 60% they grow by 1.05. This couples effective search radius to temperature, producing large exploratory jumps at high T and fine refinement near convergence. The penalty weights (λ,μ) are held fixed within each trial and supplied by the caller, which may ramp them between trials to progressively tighten feasibility enforcement. Table **??** summarises the parameters.

3.3. Outer Loop: Bisection and Polish

The SA oracle answers one question: is there a feasible configuration at this L? Finding the minimum L requires an outer bisection loop. The initial bracket is $[L_{\text{lo}}, L_{\text{hi}}]$ where $L_{\text{lo}} = L_{\text{area}} = \sqrt{NA_p}$ (the provably infeasible lower bound) and $L_{\text{hi}} = \gamma L_{\text{lo}}$ with $\gamma \in \{2.2, 2.6, 3.0\}$ depending on N. At each bisection step the SA is run at $L_{\text{mid}} = \frac{1}{2}(L_{\text{lo}} + L_{\text{hi}})$: if feasible, $L_{\text{hi}} \leftarrow L_{\text{mid}}$; otherwise $L_{\text{lo}} \leftarrow L_{\text{mid}}$. The loop terminates when the relative bracket width satisfies $(L_{\text{hi}} - L_{\text{lo}}) \leq 10^{-3} L_{\text{hi}}$, certifying L^* to a relative precision of 10^{-3}. Before each SA run at a new L, all polygon centres are rescaled by $\alpha = L_{\text{new}}/L_{\text{old}}$, a warm start that preserves structural information and reduces the time to re-establish feasibility.

After bisection, a polish stage calls the SA oracle repeatedly at the best known L_{hi} to improve configuration quality without further narrowing the container. Progress is written to disk on every improvement, and a global snapshot is flushed on `SIGTERM` so SLURM preemption loses no work. Every worker is seeded deterministically from (base seed, run ID, trial index) via SplitMix64, guaranteeing full reproducibility.

4. EXPERIMENTAL SETUP

All experiments ran on the University of Arizona HPC cluster (account `ece569`, `standard` partition). Two complementary job families were used.

The *five-node embarrassingly parallel* jobs allocated five exclusive nodes, each with 32 cores and 16 GB RAM. Worker count was $W = 30$ per node (reserving two cores for the OS), giving 150 concurrent workers. Each node queued 64 seeds via `xargs -P 30`, so 320 seeds ran per job across three concurrency waves. This pipeline produced the $N = 5$ and $N = 10$ cohorts (smoke tests, 3-minute wall time) and was the intended design for larger N.

The *single-node array* jobs ran at most two seeds concurrently and produced the $N \in \{25, 50, 100, 200\}$ results reported here: 8 array tasks at $N = 25$ (4-hour wall time, effective budget 14,100 s), 6 at $N = 50$ (4 h), 4 at $N = 100$ (4 h), and 2 at $N = 200$ (8 h, effective 28,500 s). SLURM sent `SIGTERM` 120 s before the hard kill, giving the solver time to flush its best result. No inter-worker communication occurred during any run; the global best for each N was found in a post-hoc reduction pass over all output CSVs.

5. RESULTS

Table **??** reports the best feasible side length L^* and packing efficiency $\eta = NA_p/(L^*)^2$ for each N. The area lower bound $L_{\text{area}} = \sqrt{NA_p}$ is given for reference; $\eta = 1$ would require zero wasted space, which is geometrically impossible for this non-convex shape. The column "Runs" counts workers that found at least one feasible configuration.

Table 2. Best feasible side length, packing efficiency, and feasibility rate by instance size.

N	L_{area}	L^*	η	Runs	t_{first}
5	1.108	1.472	0.567	15	5 s
10	1.568	2.056	0.581	15	6 s
25	2.478	3.315	0.559	6	8 s
50	3.504	4.652	0.567	4	46 s
100	4.956	6.785	0.533	3	53 s
200	7.009	10.104	0.481	1	19 s

Three patterns emerge. First, the number of workers reaching

feasibility drops sharply: 15 of 15 at $N = 5$ and $N = 10$, down to 6 at $N = 25$, and to a single run at $N = 200$. The SA is time-budgeted and not iteration-bounded, but the cost per SA proposal grows with N because more polygon pairs must be checked. At large N, fewer trials fit within the wall-time budget, and the trials that do run explore a configuration space of dimension $3N$ with proportionally less coverage per polygon. The configuration space at $N = 200$ has 600 degrees of freedom; the budget was calibrated on much smaller instances.

Second, best-of-best efficiency η declines monotonically, from 0.58 at small N to 0.48 at $N = 200$. Fitting $\eta(N) \approx \eta_\infty + c/\sqrt{N}$ to the $N = 50$ and $N = 200$ points gives $\eta_\infty \approx 0.40$ and $c \approx 1.22$. The scaling has a clean physical basis: wasted space due to boundary proximity grows with the container perimeter, which scales as $L \propto \sqrt{N}$, while bulk packing capacity grows with the area, scaling as $L^2 \propto N$; the relative boundary penalty therefore decays as $1/L \propto 1/\sqrt{N}$. The extrapolated limit $\eta_\infty \approx 0.40$ should be read with caution: at large N the density drop reflects solver budget limitations as much as any geometric asymptote.

Third, the median time to final best configuration was 205 s at $N = 25$ but exceeded 3,250 s at $N = 100$, out of a 14,100 s budget. Most improvement arrived at 23–26% into the run, confirming that the polish phase, not the bisection loop, produces the majority of the density gain at large N. At small N the bisection alone is nearly sufficient; at large N the bulk of the productive computation happens during extended feasibility polish. Figures **??–??** show these dynamics.

6. CONCLUSION

Choosing SAT over a binary overlap test was the key design decision: penetration depth provides the continuous energy gradient that steers SA toward feasibility from arbitrary, heavily overlapping initial configurations. The bisection loop tightens the bracket on L^* to a relative precision of 10^{-3}, and the polish stage then fills whatever wall time remains with incremental quality improvement. The experiments confirm the solver reaches feasible packings for N up to 200 within practical HPC allocations, but expose a clear bottleneck: as N grows, the time-budgeted SA visits fewer trials relative to the dimension of the search space, and the feasibility rate collapses. The natural first intervention is to scale the time budget proportionally with N.

Three directions motivate future work. A CUDA implementation targeting the P100 (compute capability 6.0) could exploit the embarrassingly parallel trial structure, with one thread block per polygon pair and triangle vertex arrays in shared memory. Parallel Tempering, which runs replicas at a geometric temperature ladder and proposes Metropolis swaps between adjacent temperatures, could seed cold-chain refinement from configurations that single-chain SA never visits, though it requires substantially longer wall times than tested here. Finally, a genetic algorithm treating full polygon layouts as chromosomes could maintain structural diversity across qualitatively different configurations simultaneously, something serial SA cannot do.

REFERENCES

[1] Kaggle. Santa 2024 competition: The polytope permutation puzzle. *kaggle.com/competitions/santa-2024*, 2024.

[2] E. D. Demaine and J. O'Rourke. *Geometric Folding Algorithms: Linkages, Origami, Polyhedra*. Cambridge University Press, 2007.

[3] J. A. Bennell and J. F. Oliveira. The geometry of nesting problems: A tutorial. *Eur. J. Oper. Res.*, 184(2):397–415, 2008.

[4] T. W. Leung, C. K. Chan, and M. D. Troutt. Application of a mixed simulated annealing-genetic algorithm heuristic for the two-dimensional orthogonal packing problem. *Eur. J. Oper. Res.*, 145(3):530–542, 2003.

[5] E. K. Burke, G. Kendall, and G. Whitwell. A new placement heuristic for the orthogonal stock-cutting problem. *Oper. Res.*, 52(4):655–671, 2004.

[6] K. Hukushima and K. Nemoto. Exchange Monte Carlo method and application to spin glass simulations. *J. Phys. Soc. Jpn.*, 65(6):1604–1608, 1996.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

Figure 1. Feasible square side length L over wall time for $N \in \{5, 10, 25, 50, 100, 200\}$ (panels a–f). Thin lines are individual worker traces; the thick line is the aggregate best-so-far envelope. The right panel for each row shows the best packing geometry found. At $N = 200$ only one worker reached feasibility, producing a single visible trace.

Figure 2. Best-so-far packing efficiency $\eta = NA_p/L^2$ versus wall time. Thin lines are individual workers; the thick line is the median across feasible workers. The plateau reflects the bisection loop locking in a bracket; improvement thereafter comes from the polish stage.

Figure 3. Wall time to final best configuration by N. Boxes are interquartile ranges; dots are individual workers. The sharp increase at $N \geq 50$ confirms that the polish stage, not bisection, produces most improvement at scale.

Figure 4. Best packing efficiency η versus N. The monotonic decline reflects both boundary geometry effects and a time budget that covers fewer trials per degree of freedom as N grows.

Figure 5. Packing efficiency η versus $1/\sqrt{N}$. The linear trend supports $\eta(N) \approx \eta_\infty + c/\sqrt{N}$ with $\eta_\infty \approx 0.40$, grounded in the container perimeter-to-area scaling argument.